

## PRESENTATION DETAILLE DE L'ETUDE DE CAS

« AutoLoc »



Plateforme de gestion de location de véhicules multi-agences

### 1. Objectifs pédagogiques

Cette étude de cas, filée sur l'ensemble des 13 séances du cours d'Architecture des Systèmes d'Information, s'appuie sur un projet unique « AutoLoc » enrichi progressivement à chaque atelier. Les étudiants observent ainsi la construction incrémentale d'une architecture logicielle complète, depuis la modélisation jusqu'à l'évaluation de la qualité et aux pistes d'amélioration continue.

**À l'issue du parcours, l'étudiant doit être capable de :**

- Identifier les styles et patrons d'architecture des systèmes d'information et situer l'écosystème Spring dans le développement d'applications Java d'entreprise (Séance 1) ;
- Générer un projet Spring Boot / Maven et créer les premières entités JPA d'AutoLoc à l'aide de Lombok (Séance 2) ;
- Concevoir un modèle de données relationnel avec Spring Data JPA (entités, associations, cascade, fetch), en mappant les associations entre entités JPA (@OneToOne, @OneToMany, @ManyToOne, @ManyToMany) et en justifiant les stratégies de cascade et de fetch retenues pour AutoLoc (Séance 3) ;
- Identifier et corriger les défauts d'une conception logicielle non maîtrisée, puis y appliquer les patrons de conception (GoF) pertinents (Strategy, Factory, Observer, Singleton) sur une version mal conçue d'AutoLoc (Séance 4) ;

- Structurer une application en couches respectant les principes SOLID et l'injection de dépendances, en mettant en place les couches Repository et Service avec Spring Data JPA (Séances 5 & 6) ;
- Exposer une API REST documentée, sécurisée par la validation des données (DTO + Bean Validation), avec Spring MVC et SpringDoc/OpenAPI d'une part (Séance 7), puis en introduisant les DTO, la validation et le refactoring Clean Code d'autre part (Séance 9) ;
- Mettre en œuvre les opérations d'affectation et de désaffectation entre entités liées, en garantissant la cohérence des associations bidirectionnelles et l'atomicité des traitements via @Transactional (Séance 8) ;
- Interroger les données avec les requêtes dérivées et JPQL, y compris agrégations et pagination, et automatiser des traitements différés avec Spring Scheduler (Séances 10 & 11) ;
- Instrumenter une application avec la programmation orientée aspect (Spring AOP), pour la journalisation, la mesure de performance et le contrôle de rôle sur les opérations sensibles d'AutoLoc (Séance 12) ;
- Évaluer la qualité et les limites d'une architecture monolithique et proposer des pistes d'évolution, à travers des métriques quantitatives et qualitatives (SonarQube) et l'analyse de la dette technique (Séance 13) ;

## 2. Présentation du projet : AutoLoc

### 2.1 Contexte métier

« AutoLoc » est une entreprise de location de véhicules disposant de plusieurs agences réparties dans différentes villes. Chaque agence gère une flotte de véhicules de catégories variées (citadine, berline, SUV, utilitaire). Les clients réservent un véhicule pour une période donnée ; à la signature, un contrat est établi et des paiements y sont rattachés. L'entreprise souhaite numériser l'ensemble du processus : réservation, contractualisation, facturation, suivi de flotte et relances automatiques, au travers d'une application back-end Spring Boot exposant une API REST.

## 2.2 Acteurs du système

| Rôle                           | Description                                                 | Droits principaux                                                                          |
|--------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Client                         | Particulier ou professionnel souhaitant louer un véhicule   | Consulter les véhicules disponibles, créer/annuler une réservation, consulter ses contrats |
| Agent d'agence                 | Employé en charge de la gestion opérationnelle d'une agence | Gérer les véhicules, valider une réservation, établir un contrat, enregistrer un paiement  |
| Responsable d'agence (Manager) | Supervise une agence et son personnel                       | Droits Agent + gestion des employés, consultation des statistiques de l'agence             |
| Administrateur                 | Administre la plateforme                                    | Gestion des agences, des catégories de véhicules, statistiques globales, configuration     |

## 2.3 Modules fonctionnels

| Module                             | Description                                                                                                                                    |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Gestion des agences & de la flotte | CRUD des agences, des véhicules et de leurs catégories ; suivi de la disponibilité et du statut (disponible, loué, en maintenance).            |
| Gestion des clients                | Inscription, mise à jour du profil, historique des réservations et des contrats.                                                               |
| Réservation                        | Recherche de véhicules disponibles par ville, catégorie et période ; création, modification et annulation de réservations.                     |
| Contractualisation & paiement      | Génération d'un contrat à la validation d'une réservation, enregistrement des paiements (acompte, solde, pénalités).                           |
| Tarification                       | Calcul du tarif selon la catégorie du véhicule, la durée, la période (semaine/week-end) et les équipements optionnels (GPS, siège bébé, etc.). |
| Tâches planifiées                  | Libération automatique des véhicules en fin de location, alertes de contrats arrivant à échéance, statistiques.                                |
| Reporting & qualité                | Statistiques d'occupation et de chiffre d'affaires par agence, indicateurs de qualité du code et de couverture de tests.                       |

### 3. Modèle de données conceptuel

Le modèle présenté ci-dessous constitue le **socle métier du projet**, construit progressivement au cours de la **Séance 2 (Atelier 1)** et de la **Séance 3 (Atelier 2)**. Il a été conçu de manière **approfondie** afin d'illustrer les différents types d'associations et mécanismes de gestion des relations entre entités abordés en cours, notamment **@OneToMany**, **@ManyToOne**, **@OneToOne**, **@ManyToMany**, ainsi que les stratégies de **Cascade** et de **Fetch**.

| Entité      | Attributs principaux                                                    | Association                                                                       | Type / Cascade-Fetch                         |
|-------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------|
| Agence      | id, nom, ville, adresse, telephone                                      | 1 Agence → N Vehicule ; 1 Agence → N Employe                                      | @OneToMany, FetchType.LAZY                   |
| Vehicule    | id, immatriculation, marque, modele, categorie, tarifJournalier, statut | N Vehicule → 1 Agence ; N Vehicule ↔ N Equipement                                 | @ManyToOne / @ManyToMany (table de jointure) |
| Equipement  | id, libelle (GPS, siège bébé, coffre de toit...)                        | N Equipement ↔ N Vehicule                                                         | @ManyToMany, FetchType.LAZY                  |
| Client      | id, nom, prenom, email, telephone, numPermis, dateInscription           | 1 Client → N Reservation                                                          | @OneToMany, cascade = PERSIST                |
| Employe     | id, nom, prenom, role (AGENT/MANAGER)                                   | N Employe → 1 Agence                                                              | @ManyToOne, FetchType.LAZY                   |
| Reservation | id, dateDebut, dateFin, statut                                          | N Reservation → 1 Client ; N Reservation → 1 Vehicule ; 1 Reservation → 1 Contrat | @ManyToOne ×2, @OneToOne                     |
| Contrat     | id, dateSignature, montantTotal, valide                                 | 1 Contrat → 1 Reservation ; 1 Contrat → N Paiement                                | @OneToOne, @OneToMany cascade = ALL          |
| Paiement    | id, montant, datePaiement, modePaiement                                 | N Paiement → 1 Contrat                                                            | @ManyToOne, FetchType.LAZY                   |
| Maintenance | id, dateDebut, dateFin, description                                     | N Maintenance → 1 Vehicule                                                        | @ManyToOne, cascade = PERSIST                |

#### Remarque

La relation **Contrat → Paiement** en **cascade = ALL** permet d'illustrer la suppression en cascade (suppression d'un contrat ⇒ suppression de ses paiements), tandis que **Agence → Vehicule** reste volontairement sans cascade de suppression (on ne supprime pas un véhicule en supprimant une agence).

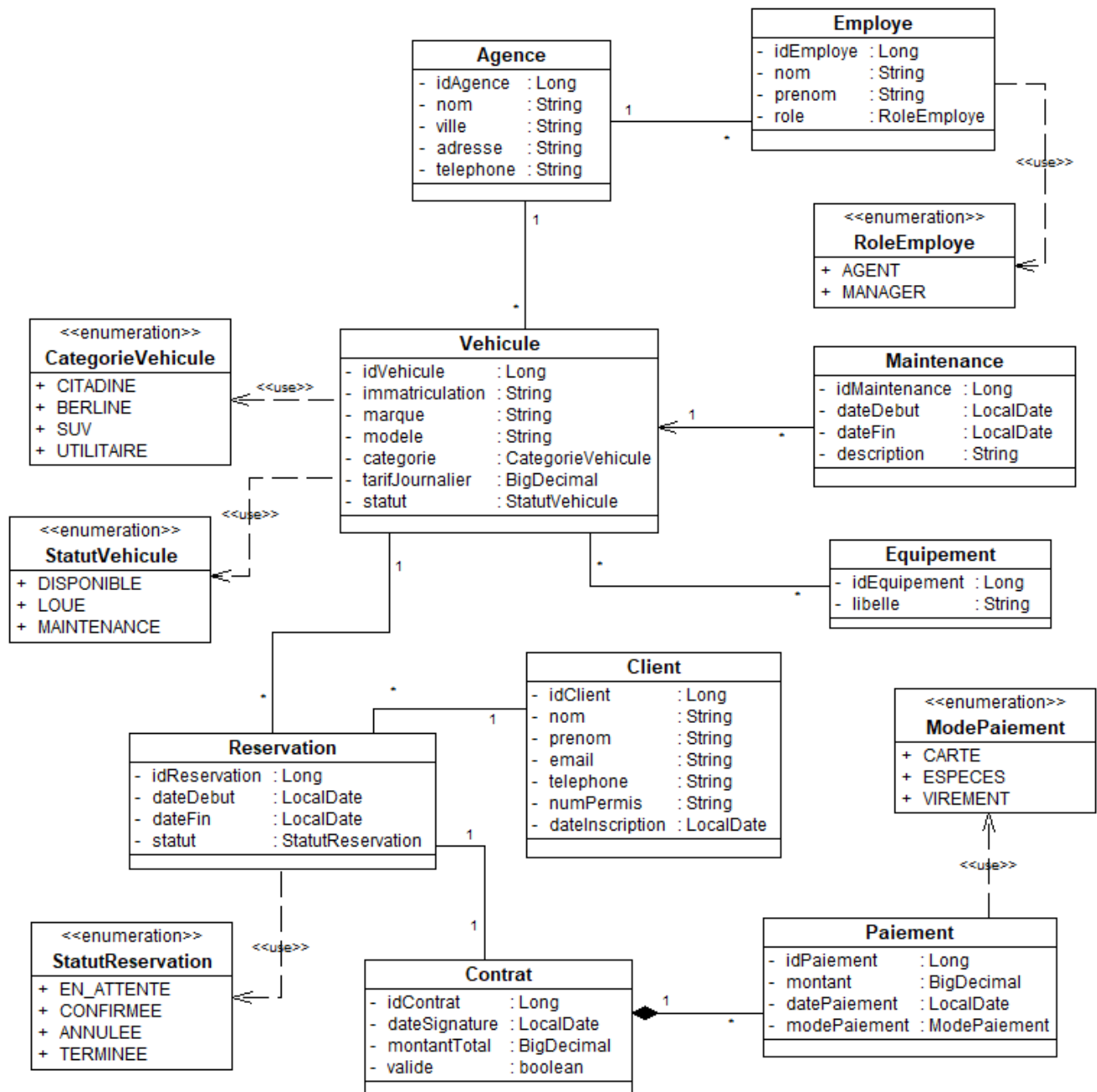


Figure 1. Modèle Entités/Associations - Etude de cas « AutoLoc »

Ce modèle sera implémenté progressivement : les entités seront développées lors de l'Atelier 1, puis leurs associations lors de l'Atelier 2 (Séance 3), avec une justification des choix de cascade et de fetch pour chacune d'entre elles.

## 4. Architecture retenue

L'application cible adopte un **style architectural monolithique**, structuré selon le **patron d'architecture en couches (Layered Architecture)**. Cette organisation repose sur une séparation

claire des responsabilités entre les différentes couches de l'application et s'appuie sur plusieurs **patterns de conception**.

Le tableau ci-dessous décrit, pour chaque couche, son **rôle**, le **contenu attendu**, le **package cible** ainsi que des **exemples de classes du projet « AutoLoc »**. Il constitue un guide de référence permettant aux apprenants d'identifier précisément l'emplacement et la responsabilité du code développé au cours de chaque atelier.

| Couche                    | Package            | Contenu de la solution                                                                                               |
|---------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Presentation (API)</b> | web.controller     | VehiculeController, ClientController, ReservationController, ContratController, StatistiqueController, OpenApiConfig |
| <b>Transfert (DTO)</b>    | web.dto            | VehiculeDTO, ReservationRequestDTO/ResponseDTO, VehiculeMapper                                                       |
| <b>Service (metier)</b>   | service            | IReservationService/Impl, ITarificationService + strategies                                                          |
| <b>Persistence</b>        | repository         | IVehiculeRepository, IClientRepository, IReservationRepository                                                       |
| <b>Domaine</b>            | domain             | Vehicule, Client, Reservation, CategorieVehicule, StatutVehicule                                                     |
| <b>Transverse</b>         | aspect / scheduler | LoggingAspect, PerformanceAspect, ContratEcheanceScheduler                                                           |

Une application Spring Boot est souvent organisée en couches. Chaque couche a un rôle bien précis et ne parle qu'à ses voisins directes. Ça rend le code plus facile à comprendre, à tester et à faire évoluer.

### 1. Couche présentation (les contrôleurs)

C'est la porte d'entrée de l'application. Elle reçoit les requêtes des utilisateurs (via HTTP) et leur répond. Elle ne manipule pas directement les données internes de l'application, mais des objets simplifiés appelés DTO, pour ne rien exposer d'inutile ou de sensible.

### 2. Couche métier (les services)

C'est le cerveau de l'application. Elle contient les règles et la logique : que faire, dans quel ordre, sous quelles conditions. C'est ici qu'on gère par exemple une réservation ou un calcul de prix.

### 3. Couche transversale (AOP)

Certaines tâches concernent toute l'application, comme garder une trace des actions (logs) ou gérer les erreurs. Plutôt que de répéter ce code partout, on le centralise à part. C'est un peu comme un service de sécurité qui surveille tout le bâtiment sans faire partie d'un étage en particulier.

#### **4. Couche domaine (les objets métier)**

Elle représente les "choses" que manipule l'application : un véhicule, une réservation, un client. Ce sont les briques de base du métier, avec leurs propres règles.

#### **5. Couche persistance (l'accès aux données)**

Elle s'occupe de sauvegarder et de récupérer les données dans la base de données, sans que le reste de l'application ait besoin de savoir comment.

#### **6. Base de données**

C'est là que les informations sont stockées de façon durable, avec des règles pour garantir qu'elles restent correctes et cohérentes.

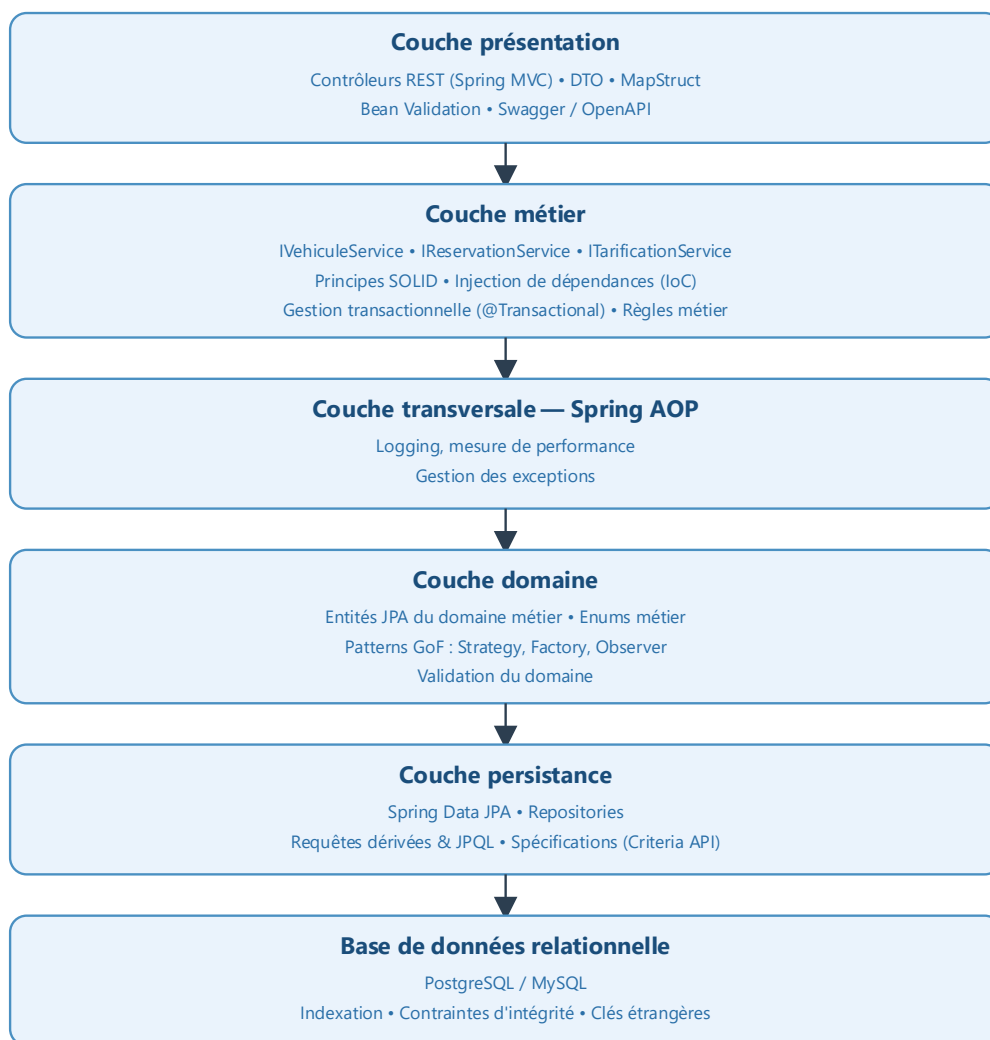


Figure 2. Architecture retenue

## 5. Stack technique recommandée

| Catégorie         | Outils / Bibliothèques                                                 |
|-------------------|------------------------------------------------------------------------|
| Langage / Build   | Java 17+, Maven                                                        |
| Framework         | Spring Boot, Spring Data JPA, Spring MVC, Spring AOP, Spring Scheduler |
| Base de données   | MySQL ou PostgreSQL en développement, H2 en mémoire pour les tests     |
| Productivité      | Lombok, SLF4J/Logback, MapStruct (mapping DTO, optionnel)              |
| Documentation API | springdoc-openapi (Swagger UI)                                         |
| Qualité           | SonarLint (IDE), SonarQube (optionnel, séance 13)                      |
| Outillage         | Git/GitHub, Postman, IntelliJ IDEA                                     |